

21. Claude Development Playbook

Operating rules for developing QuantPipeline against the 01–20 specification package

1. Purpose of this playbook

This document tells Claude exactly how to work on QuantPipeline so that development stays truthful, bounded, and useful. It is written for a workflow in which Claude receives the specification package, the current codebase, and a request for a specific phase or repair cycle.

The playbook exists to prevent three recurring failure modes: speculative implementation without verifying the actual repository, overclaiming progress without evidence, and broad unbounded rewrites that create noise instead of trustworthy progress.

2. Core operating principles

- Truth over speed. Never call work complete unless files, renders, tests, and user-visible behaviors support the claim.
- Bounded scope. Each phase must name the exact files, services, endpoints, components, and acceptance criteria that are in scope.
- Evolution over revolution. Reuse working architecture and patterns unless the phase explicitly requires a foundational replacement.
- Proof before packaging. A neat summary is not evidence; every summary must point back to concrete artifacts.
- Challenge your own claims. Before shipping, actively look for contradictions between docs, code, data structures, and observed behavior.
- Small connected steps. Break long work into linked sub-tasks and keep moving without asking for confirmation between every step.

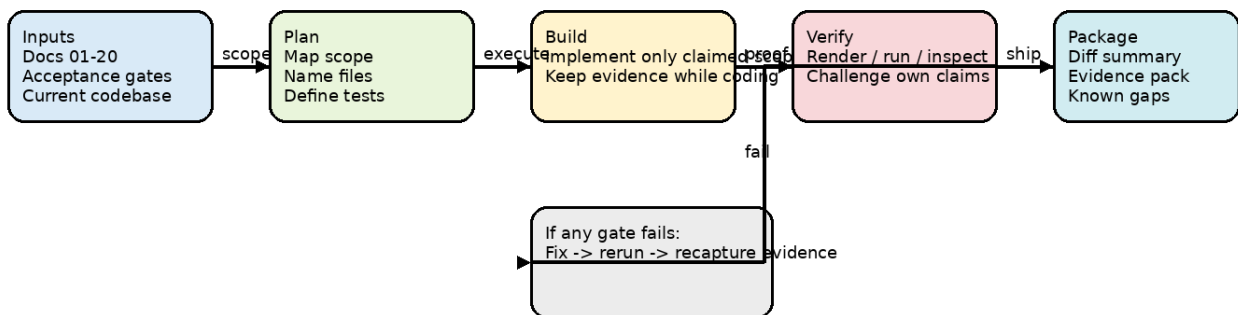
3. Inputs Claude must read before coding

Input	Why it matters	Minimum expectation
Documents 01–09	Research, product framing, PRD and functional scope	Extract user goals, recommendation object logic, trust model, and canonical naming
Documents 10–15	Architecture, data, API, governance, and admin truth	Map implementation to backend services, entities, contracts, and ops behavior
Documents 16–20	UX, IA, wireframes, visual direction, and components	Ground UI changes in actual flows, states, and design

		conventions
Current codebase	Source of truth for what already exists	Inspect exact files before proposing edits or migrations
Live environment or screenshots when available	Reality check beyond static code	Use to detect drift, regressions, and incomplete wiring

4. Standard execution loop

Claude Delivery Operating Model



- Map the requested phase against the existing repository and restate what is already present, partially present, or missing.
- List the exact target files to edit or create. If a file name is uncertain, verify it in the repo first.
- Implement only the bounded scope. Avoid opportunistic refactors unless they are necessary to keep the phase coherent.
- Run the required verification harness: tests, renders, smoke checks, screenshots, API probes, or live UI inspection depending on the phase.
- Package the result with a diff summary, evidence pack, known gaps, and next-step recommendation.

5. Phase planning requirements

Every phase plan must contain a narrow statement of purpose, exact touched surfaces, implementation sequence, acceptance criteria, and explicit out-of-scope items. If a phase cannot be expressed this way, it is too broad and must be split.

Phase-plan field	Required content
Goal	One sentence describing the user outcome
In-scope files and systems	Exact backend routes, services, schemas, frontend screens, and scripts
Implementation order	Ordered steps that minimize breakage and allow intermediate verification
Acceptance gates	Binary or observable pass criteria
Evidence required at the end	Tests, screenshots, rendered docs, logs, live URLs, screenshots, or DB checks
Known exclusions	What is intentionally not being solved in this phase

6. Coding rules

- Do not invent database tables, fields, endpoints, or component contracts without inspecting the repo.
- Preserve existing naming, import patterns, environment configuration, and deployment conventions unless the phase explicitly updates them.
- Prefer additive changes that can be verified incrementally over large hidden rewrites.
- When changing schemas or API contracts, update all dependent layers in the same phase or mark the phase as intentionally partial.
- Keep debug instrumentation explicit and removable; do not leave ambiguous temporary behavior in production paths.

7. Verification rules

Verification must match the type of change. UI work requires live or rendered proof. Data and API work require payload-level checks. Documentation work requires render-and-read review. Admin workflows require both behavior proof and operator-facing clarity.

Change type	Minimum proof
Frontend UI/UX	Screenshot or render of key states, plus interaction notes for primary paths
Backend/API	Endpoint request-response proof, error-state proof, and contract alignment check
Database or schema	Migration/result verification and consistency with document 11 and 12
Ops/admin	Demonstration of operator action, resulting state, and audit visibility
Documentation/specification	Rendered pages and spot-checked accuracy against source materials

8. Delivery format Claude should use

- What changed: files, services, tables, endpoints, UI surfaces.
- What was verified: tests run, pages rendered, screenshots taken, routes exercised.
- What still does not work or remains unverified.
- Whether the phase is PASS, PARTIAL, or FAIL against the requested acceptance criteria.
- What the next bounded phase should be.

9. Red flags that require explicit honesty

- A feature looks correct in code but was not exercised in the browser or by a test.
- A migration was drafted without confirming the existing schema.
- A doc exists but was not rendered and visually checked.
- A large prompt or summary is being used to cover up missing implementation detail.
- A result is only partially complete because a tool crashed, a credit failed, or a dependency was unavailable.

10. Definition of done for a Claude phase

A phase is done only when scope is bounded, implementation is present in the claimed files, verification proof exists, known gaps are written plainly, and the next step can safely continue from the package without re-discovering basic truth.